

Week9 - CH05. 실습

09. 회귀 실습 - 자전거 대여 수요 예측

```
from google.colab import files
uploaded = files.upload()
```

파일 선택 bike_train.csv

- **bike_train.csv**(text/csv) - 648353 bytes, last modified: 2025. 5. 4. - 100% done
Saving bike_train.csv to bike_train.csv

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

import warnings
warnings.filterwarnings('ignore', category=RuntimeWarning)

bike_df = pd.read_csv('bike_train.csv')
print(bike_df.shape)
bike_df.head()
```

(10886, 12)

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81	0.0	3	1
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80	0.0	8	3
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635	80	0.0	5	2
3	2011-01-01 03:00:00	1	0	0	1	9.84	14.395	75	0.0	3	1
4	2011-01-01 04:00:00	1	0	0	1	9.84	14.395	75	0.0	0	

- 해당 데이터 세트는 10886개의 레코드와 12개의 칼럼으로 구성

```
bike_df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   datetime    10886 non-null  object
1   season      10886 non-null  int64
2   holiday     10886 non-null  int64
3   workingday  10886 non-null  int64
4   weather     10886 non-null  int64
5   temp        10886 non-null  float64
6   atemp       10886 non-null  float64
7   humidity    10886 non-null  int64
8   windspeed   10886 non-null  float64
9   casual      10886 non-null  int64
10  registered  10886 non-null  int64
11  count       10886 non-null  int64
dtypes: float64(3), int64(8), object(1)
memory usage: 1020.7+ KB
```

- 10886개의 row 데이터 중 Null 데이터 없음
- 대부분의 칼럼이 int 또는 float형, datetime만 object형
 - → datetime 가공이 필요함
 - 판다스는 문자열을 datetime 타입으로 변환하는 apply 메서드 제공

```
제한된 코드에 라이선스가 적용될 수 있습니다. | sejinko/practice
bike_df['datetime'] = bike_df.datetime.apply(pd.to_datetime)
```

```
bike_df['year'] = bike_df.datetime.apply(lambda x : x.year)
bike_df['month'] = bike_df.datetime.apply(lambda x : x.month)
bike_df['day'] = bike_df.datetime.apply(lambda x : x.day)
bike_df['hour'] = bike_df.datetime.apply(lambda x : x.hour)
bike_df.head(3)
```

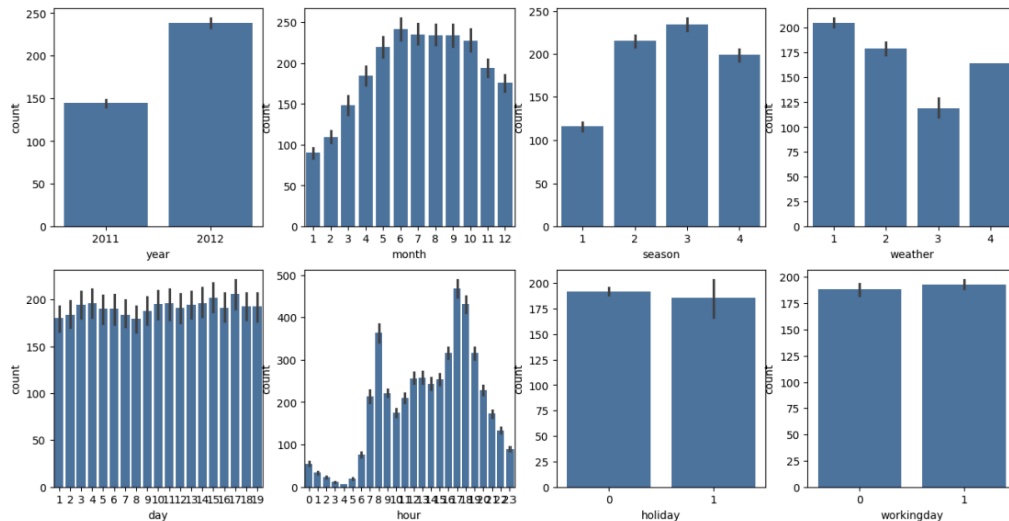
	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81	0.0	3	1
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80	0.0	8	3
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635	80	0.0	5	2

```
drop_columns = ['datetime', 'casual', 'registered']
bike_df.drop(drop_columns, axis=1, inplace=True)
```

```
fig, axs = plt.subplots(figsize=(16, 8), ncols=4, nrows=2)
cat_features = ['year', 'month', 'season', 'weather', 'day', 'hour', 'holiday', 'workingday']

for i, feature in enumerate(cat_features):
    row = int(i/4)
    col = i%4

    sns.barplot(x=feature, y='count', data=bike_df, ax=axs[row][col])
```



- 새롭게 year, month, day, hour 칼럼 추가
- datetime 칼럼 삭제
- casual + registered = count이므로 count만 남기고 삭제

```
from sklearn.metrics import mean_squared_error, mean_absolute_error
def rmsle(y, pred):
    log_y = np.log1p(y)
    log_pred = np.log1p(pred)
    squared_error = (log_y - log_pred) ** 2
    rmsle = np.sqrt(np.mean(squared_error))
    return rmsle

def rmse(y, pred):
    return np.sqrt(mean_squared_error(y, pred))

def evaluate_regr(y, pred):
    rmsle_val = rmsle(y, pred)
    rmse_val = rmse(y, pred)

    mae_val = mean_absolute_error(y, pred)
    print('RMSLE: {0:.3f}, RMSE: {1:.3f}, MAE: {2:.3f}'.format(rmsle_val, mae_val))
```

- 회귀 모델을 데이터 세트에 적용하여 예측 성능 측정
- 사이킷런은 RMSLE 제공하지 않으므로 직접 만들

```
from sklearn.metrics import mean_squared_log_error
mse = mean_squared_error

def rmsle(y, pred):
    msle = mean_squared_log_error(y, pred)
    msle = np.sqrt(mse)
    return msle
```

- rmsle 구할 때 넘파이의 log()함수를 이용하거나 사이킷런의 meansquared error 이용 가능
 - But, 오버플로/언더플로 오류 발생할 수 있음

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression, Ridge, Lasso

y_target = bike_df['count']
X_features = bike_df.drop(['count'], axis=1, inplace=False)

X_train, X_test, y_train, y_test = train_test_split(X_features, y_target, test_size=0.3, random_state=0)

lr_reg = LinearRegression()
lr_reg.fit(X_train, y_train)
pred = lr_reg.predict(X_test)
```

- 데이터 세트 결괏값이 정규 분포로 돼 있는지 확인,
- 카테고리형 회귀 모델의 경우 원-핫 인코딩으로 feature 인코딩

```
def get_top_error_data(y_test, pred, n_tops = 5):
    result_df = pd.DataFrame(y_test.values, columns=['real_count'])
    result_df['predicted_count'] = np.round(pred)
    result_df['diff'] = np.abs(result_df['real_count'] - result_df['predicted_count'])

    print(result_df.sort_values('diff', ascending=False)[:n_tops])

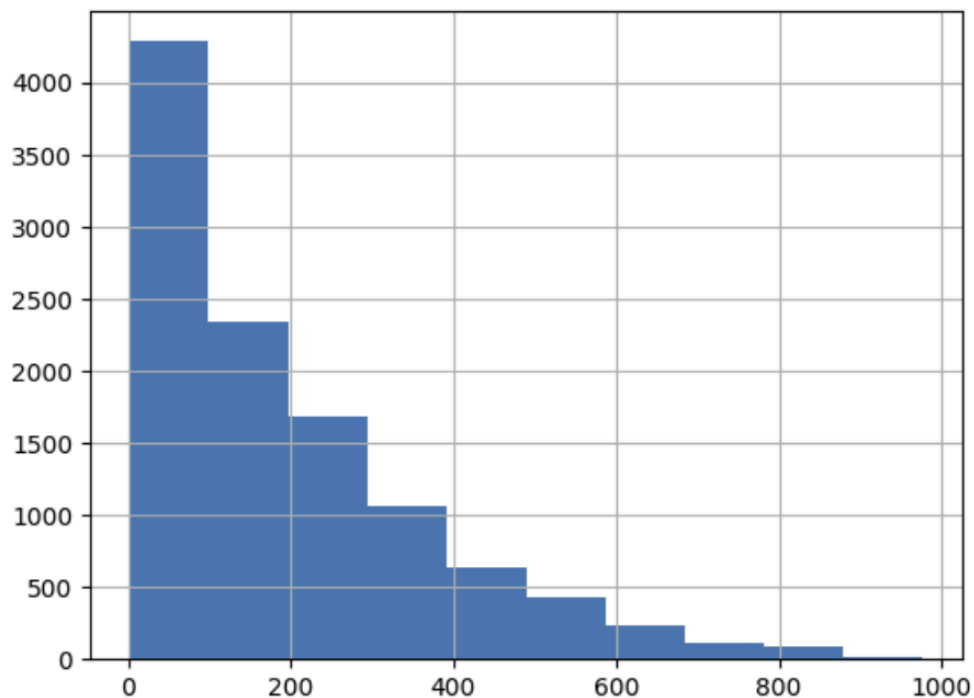
get_top_error_data(y_test, pred, n_tops=5)
```

	real_count	predicted_count	diff
1618	890	322.0	568.0
966	884	327.0	557.0
3151	798	241.0	557.0
412	745	194.0	551.0
2817	856	310.0	546.0

- 오류 값이 가장 큰 순으로 5개만 확인
- 예측 오류가 꽤 큼
 - 회귀에서 큰 예측 오류 발생할 경우 Target 값의 분포가 왜곡된 형태인지 확인
 - Target 값의 분포는 정규 분포 형태가 가장 좋음
 - 왜곡된 경우에는 회귀 예측 성능이 저하되는 경우가 발생하기 때문

```
y_target.hist()
```

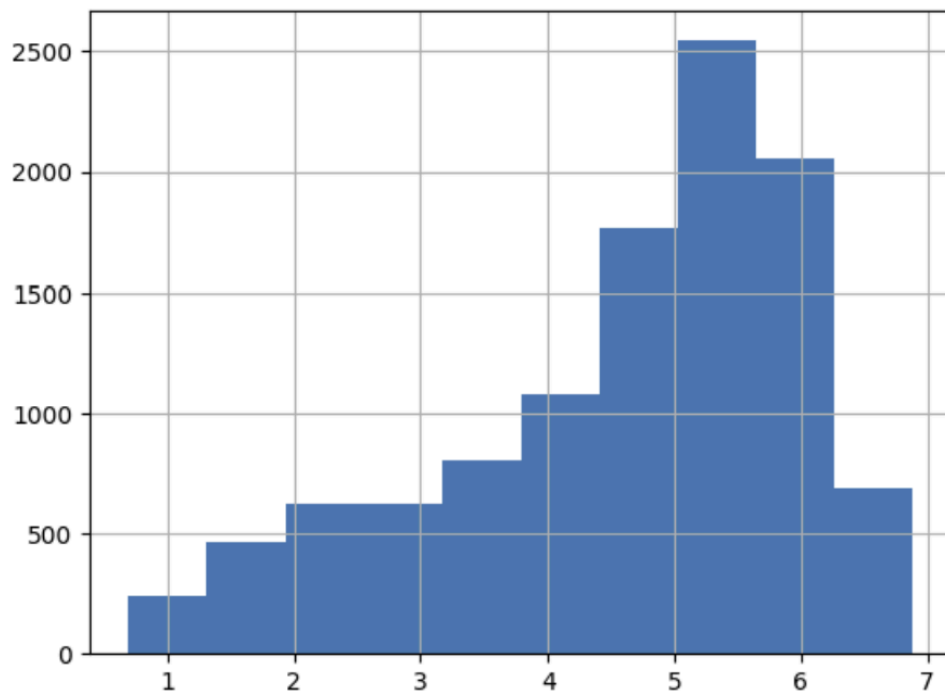
<Axes: >



- 판다스 DataFrame의 hist()를 이용해 Target 값인 count 칼럼이 정규 분포인지 확인
- 0-200 사이에 왜곡된 것을 볼 수 있음
- 정규 분포 형태로 바꾸는 가장 일반적인 방법은 로그를 적용하여 변환하는 것

```
y_log_transform = np.log1p(y_target)
y_log_transform.hist()
```

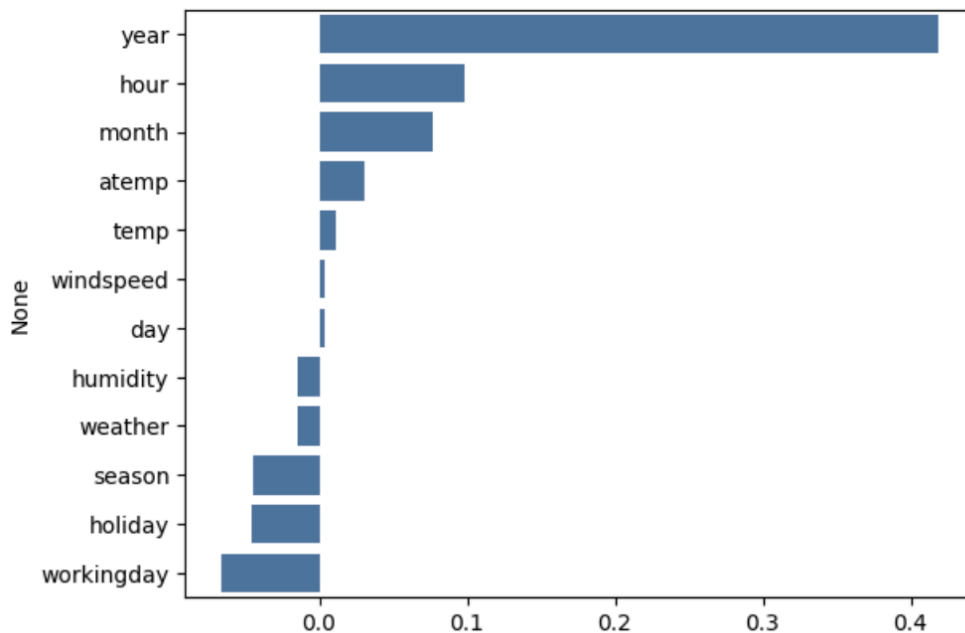
<Axes: >



- 넘파이의 log1p()를 이용
- 변경된 Target 값을 기반으로 학습하고 예측한 값은 다시 expm1() 함수를 적용해 원래 scale 값으로 원상 복구
- 변환 후 정규 분포는 아니어도 왜곡 많이 나아짐

```
coef = pd.Series(lr_reg.coef_, index=X_features.columns)
coef_sort = coef.sort_values(ascending=False)
sns.barplot(x=coef_sort.values, y=coef_sort.index)
```

<Axes: ylabel='None'>



- 개별 feature들의 인코딩을 적용
- 각 feature의 회귀 계숫값 시각화
- year, hour, month 모두 숫자 값으로 표현되지만 모두 category형 feature
- 사이킷런은 카테고리만을 위한 데이터 타입 없으며 모두 숫자로 변환해야 함
- 선형 회귀에서는 이러한 feature 인코딩에 원-핫 인코딩 적용하여 변환해야 됨

```
X_features_ohc = pd.get_dummies(X_features, columns = ['year', 'month', 'day', 'hour', 'holiday', 'workingday'])
```

제한된 코드에 라이선스가 적용될 수 있습니다. [ragu6963/kfq_python_machine_learning]
X_train, X_test, y_train, y_test = train_test_split(X_features_ohc, y_target_log, test_size=0.3, random_state=

```
def get_model_predict(model, X_train, X_test, y_train, y_test, is_exp1=False):
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    if is_exp1:
        y_test = np.exp1(y_test)
        pred = np.exp1(pred)
    print('###', model.__class__.__name__, '###')
```

```
lr_reg = LinearRegression()
ridge_reg = Ridge(alpha=10)
lasso_reg = Lasso(alpha=0.01)
```

```
for model in [lr_reg, ridge_reg, lasso_reg]:
    get_model_predict(model, X_train, X_test, y_train, y_test, is_exp1=True)
```

- 선형 회귀 모델 모두 학습해 예측 성능 확인
 - get_model_predict() 함수 만들
- 원-핫 인코딩 적용 후 예측 성능 많이 향상 되

```
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor

rf_reg = RandomForestRegressor(n_estimators=500)
gbm_reg = GradientBoostingRegressor(n_estimators=500)
xgb_reg = XGBRegressor(n_estimators=500)
lgbm_reg = LGBMRegressor(n_estimators=500)

for model in [rf_reg, gbm_reg, xgb_reg, lgbm_reg]:
    get_model_predict(model, X_train.values, X_test.values, y_train.values, y_test.values, is_exp1=True)

### RandomForestRegressor ###
```

- 회귀 트리 이용하여 회귀 예측 수행
- 앞에서 적용한 Target 값의 로그 변환된 값과 원-핫 인코딩된 feature 데이터 세트를 그대로 이용해 성능 평가
- XGBoost의 경우 DataFrame이 학습/테스트 데이터로 입력될 경우 버전에 따라 오류 가능
 - 학습/테스트 데이터를 DataFrame의 values 속성을 이용해 넘파이 ndarray로 변환

10. 회귀 실습 - 캐글 주택 가격: 고급 회귀 기법


```
from google.colab import files
uploaded = files.upload()
```

파일 선택 house_price.csv

- **house_price.csv(text/csv)** - 460676 bytes, last modified: 2025. 5. 4. - 100% done

Saving house_price.csv to house_price.csv

```
import warnings
warnings.filterwarnings('ignore')
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

house_df_org = pd.read_csv('house_price.csv')
house_df = house_df_org.copy()
house_df.head(3)
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...

3 rows × 81 columns

- Target 값은 맨 마지막 칼럼인 SalePrice

```
print('데이터 세트의 Shape:', house_df.shape)
print('\n전체 피처의 type\n', house_df.dtypes.value_counts())
isnull_series = house_df.isnull().sum()
print('\nNull 칼럼과 그 건수:\n', isnull_series[isnull_series>0].sort_values(ascending=False))
```

데이터 세트의 Shape: (1460, 81)

전체 피처의 type

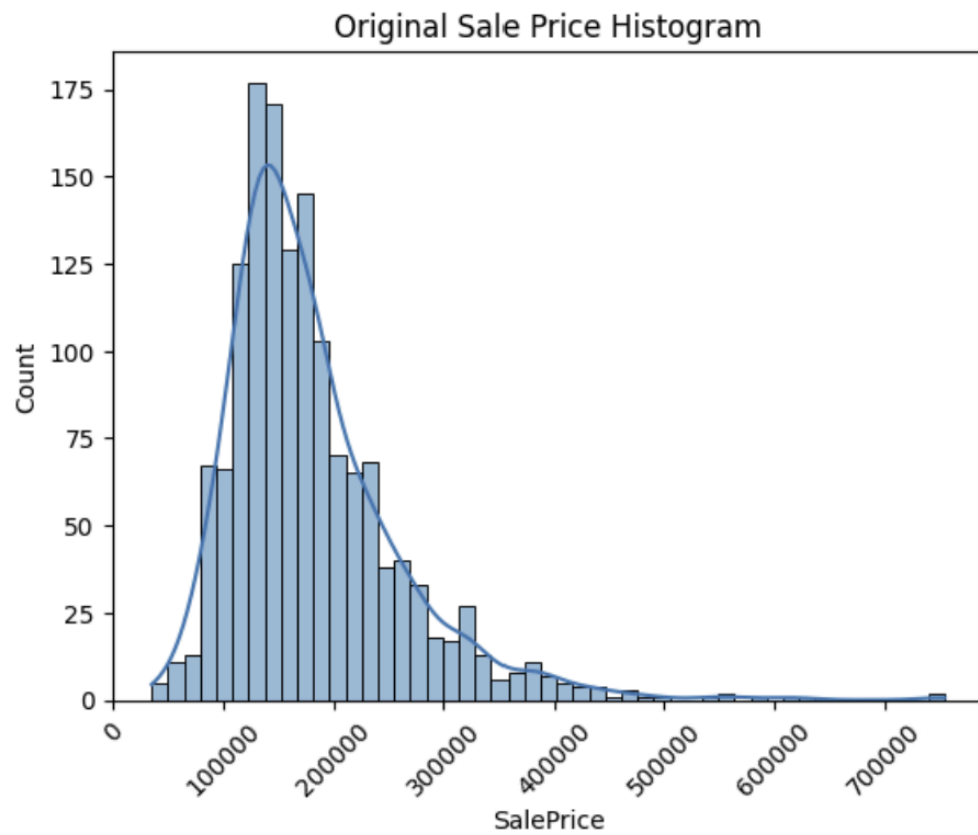
```
object      43
int64       35
float64      3
Name: count, dtype: int64
```

Null 칼럼과 그 건수:

```
PoolQC      1453
MiscFeature  1406
Alley       1369
Fence       1179
MasVnrType   872
FireplaceQu  690
LotFrontage  259
GarageType    81
GarageYrBlt   81
GarageFinish  81
GarageQual    81
GarageCond    81
BsmtExposure  38
BsmtFinType2  38
BsmtQual      37
BsmtCond      37
BsmtFinType1  37
MasVnrArea     8
Electrical     1
dtype: int64
```

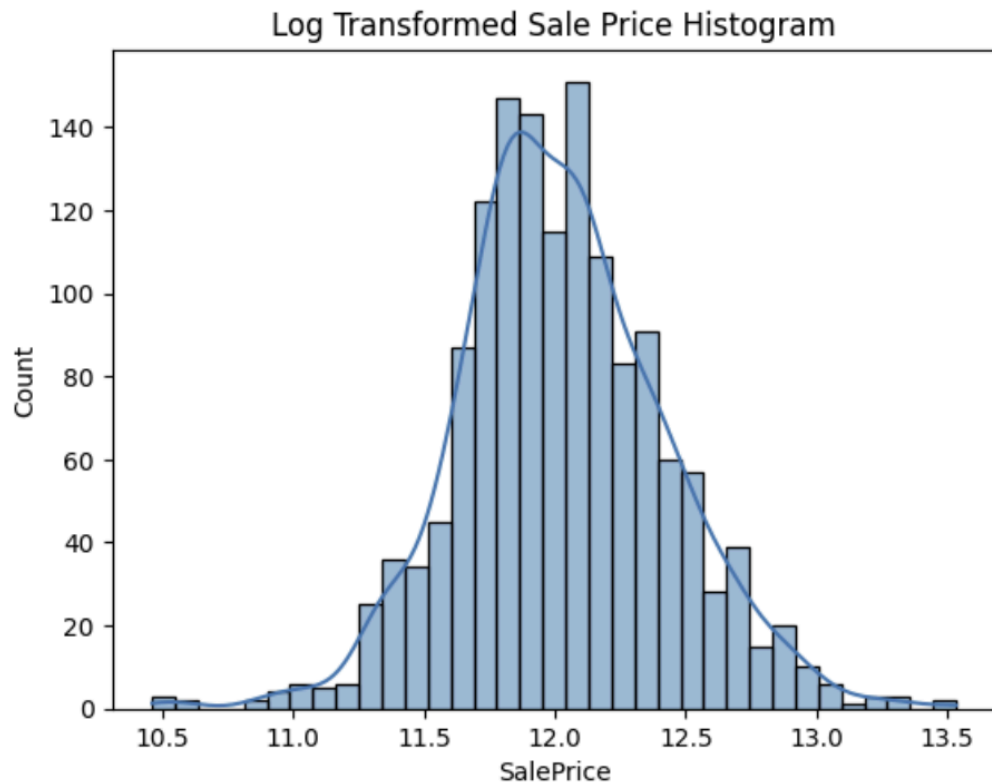
- 데이터 세트의 전체 크기와 칼럼의 타입, Null이 있는 칼럼과 그 건수를 내림차순으로 출력
- 데이터 세트는 1460개의 레코드와 81개의 feature로 구성
- feature의 타입은 숫자형, 문자형 섞여 있음
- PoolQC, MiscFeature, Alley, Fence는 1000개 넘는 데이터가 Null
 - Null 값 너무 많은 feature는 드롭

```
plt.title('Original Sale Price Histogram')
plt.xticks(rotation=45)
sns.histplot(house_df['SalePrice'], kde=True)
plt.show()
```



- 값의 분포도 확인
- 왼쪽으로 치우친 형태, 정규 분포에서 벗어남
- 정규 분포가 아닌 결괏값을 정규 분포 형태로 변환하기 위해 로그 변환 적용

```
plt.title('Log Transformed Sale Price Histogram')
log_SalePrice = np.log1p(house_df['SalePrice'])
sns.histplot(log_SalePrice, kde=True)
plt.show()
```



- 넘파이의 `log1p()` 이용하여 로그 변환한 결괏값을 기반으로 학습, 예측 시에는 다시 결괏값을 `expm1()`으로 추후 환원
- SalePrice를 로그 변환해 정규 분포 형태로 결괏값이 분포함을 확인

```

original_SalePrice = house_df['SalePrice']
house_df['SalePrice'] = np.log1p(house_df['SalePrice'])

house_df.drop(['Id', 'PoolQC', 'MiscFeature', 'Alley', 'Fence', 'FireplaceQu'], axis=1, inplace=True, errors='ignore')

house_df.fillna(house_df.mean(numeric_only=True), inplace=True)

null_column_count = house_df.isnull().sum()[house_df.isnull().sum()>0]
print('## Null 피처의 Type:\n', house_df.dtypes[null_column_count.index])

## Null 피처의 Type:
MasVnrType      object
BsmtQual        object
BsmtCond        object
BsmtExposure    object
BsmtFinType1    object
BsmtFinType2    object
Electrical      object
GarageType      object
GarageFinish    object
GarageQual      object
GarageCond      object
dtype: object

```

- SalePrice를 로그 변환 뒤 DataFrame에 반영
- Id는 단순 식별자이므로 삭제
- LotFrontage는 Null 값이 259개, 평균값으로 대체
- 나머지 Null feature도 Null 값 많지 않으므로 숫자형의 경우 평균값으로 대체
- DataFrame 객체의 mean() 메서드는 자동으로 숫자형 칼럼만 추출해 칼럼별 평균값을 Series 객체로 반환

```

print('get_dummies() 수행 전 데이터 Shape:', house_df.shape)
house_df_ohe = pd.get_dummies(house_df)
print('get_dummies() 수행 후 데이터 Shape:', house_df_ohe.shape)

null_column_count = house_df_ohe.isnull().sum()[house_df_ohe.isnull().sum()>0]
print('## Null 피처의 Type:\n', house_df_ohe.dtypes[null_column_count.index])

get_dummies() 수행 전 데이터 Shape: (1460, 75)
get_dummies() 수행 후 데이터 Shape: (1460, 270)
## Null 피처의 Type:
Series([], dtype: object)

```

- 문자형 feature 제외하고는 Null 값 없음
- 문자형 feature는 모두 원-핫 인코딩으로 변환
 - 판다스의 get_dummies() 이용
- 원-핫 인코딩 이후 feature가 74개에서 271개로 증가

- Null 값을 가진 feature는 이제 존재하지 않음

```
def get_rmse(model):
    pred = model.predict(X_test)
    mse = mean_squared_error(y_test, pred)
    rmse = np.sqrt(mse)
    print(model.__class__.__name__, '로그 변환된 RMSE : ', np.round(rmse, 3))
    return rmse

def get_rmses(models):
    rmses = []
    for model in models:
        rmse = get_rmse(model)
        rmses.append(rmse)
    return rmses
```

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

y_target = house_df_ohe['SalePrice']
X_features = house_df_ohe.drop('SalePrice', axis=1, inplace=False)
X_train, X_test, y_train, y_test = train_test_split(X_features, y_target, test_size=0.2, random_state=156)

lr_reg = LinearRegression()
lr_reg.fit(X_train, y_train)
ridge_reg = Ridge()
ridge_reg.fit(X_train, y_train)

lasso_reg = Lasso()
lasso_reg.fit(X_train, y_train)

models = [lr_reg, ridge_reg, lasso_reg]
get_rmses(models)
```

```
LinearRegression 로그 변환된 RMSE : 0.132
Ridge 로그 변환된 RMSE : 0.127
Lasso 로그 변환된 RMSE : 0.176
[np.float64(0.13183184688250857),
 np.float64(0.1274058283626616),
 np.float64(0.17628250556471403)]
```

- get_rmse(model)은 단일 모델의 RMSE 값을 반환
- get_rmses(models)는 get_rmse()를 이용해 여러 모델의 RMSE 값을 반환
- 라쏘 회귀의 경우 회귀 성능이 타 회귀 방식보다 많이 떨어짐
 - 최적 하이퍼 파라미터 튜닝 필요
- feature별 회귀 계수를 시각화, 모델 별로 어떤 feature의 회귀 계수로 구성되는지 확인

```
def get_top_bottom_coef(model, n=10):
    coef = pd.Series(model.coef_, index=X_features.columns)

    coef_high = coef.sort_values(ascending=False).head(n)
    coef_low = coef.sort_values(ascending=False).tail(n)
    return coef_high, coef_low
```

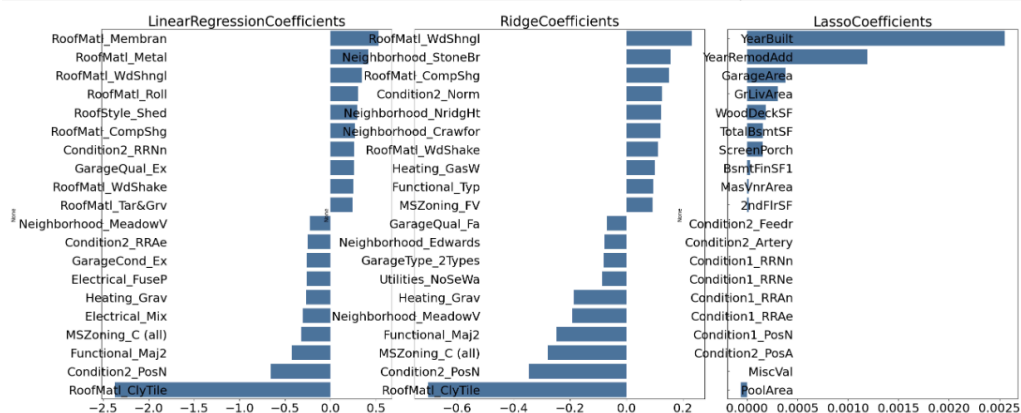
- 생성한 get_top_vottom_coef(model, n=10) 함수를 이용해 모델별 회귀 계수를 시각화
- 시각화 위한 함수로 visualize_coefficient(models)를 생성
- 해당 함수는 list 객체로 모델을 입력받아 모델별로 회귀 계수 상위 10개, 하위 10개를 추출해 가로 막대 그래프 형태로 출력

```
def visualize_coefficient(models):
    fig, axs = plt.subplots(figsize=(24, 10), nrows=1, ncols=3)
    fig.tight_layout()

    for i_num, model in enumerate(models):
        coef_high, coef_low = get_top_bottom_coef(model)
        coef_concat = pd.concat([coef_high, coef_low])

        axs[i_num].set_title(model.__class__.__name__+'Coefficients', size=25)
        axs[i_num].tick_params(axis='y', direction='in', pad=-120)
        for label in (axs[i_num].get_xticklabels() + axs[i_num].get_yticklabels()):
            label.set_fontsize(22)
        sns.barplot(x=coef_concat.values, y=coef_concat.index, ax=axs[i_num])

    models = [lr_reg, ridge_reg, lasso_reg]
    visualize_coefficient(models)
```



- 모델별 회귀 계수에 OLS 기반의 LinearRegression과 Ridge의 경우 회귀 계수가 매우 유사한 형태로 분포

- 라쏘는 전체적으로 회귀 계수 값이 매우 작음
 - 그 중 YearBuilt가 가장 큼
 - 다른 feature의 회귀 계수는 너무 작음
- 라쏘는 다른 두 모델과 다른 회귀 계수 형태 보임

```
from sklearn.model_selection import cross_val_score

def get_avg_rmse_cv(models):

    for model in models:

        rmse_list = np.sqrt(-cross_val_score(model, X_features, y_target, scoring="neg_mean_squared_error", cv
        rmse_avg = np.mean(rmse_list)
        print('\n{0} CV RMSE 값 리스트: {1}'.format(model.__class__.__name__, np.round(rmse_list, 3)))
        print('{0} CV 평균 RMSE 값: {1}'.format(model.__class__.__name__, np.round(rmse_avg, 3)))

models = [LinearRegression(), Ridge(), Lasso()]
get_avg_rmse_cv(models)
```

LinearRegression CV RMSE 값 리스트: [0.135 0.165 0.167 0.111 0.198]
LinearRegression CV 평균 RMSE 값: 0.155

Ridge CV RMSE 값 리스트: [0.117 0.154 0.142 0.117 0.189]
Ridge CV 평균 RMSE 값: 0.144

Lasso CV RMSE 값 리스트: [0.161 0.204 0.177 0.181 0.265]
Lasso CV 평균 RMSE 값: 0.198

```
제한된 코드에 라이선스가 적용될 수 있습니다. | Yanghojun/yanghojun.github.io
from sklearn.model_selection import GridSearchCV

def print_best_params(model, params):
    grid_model = GridSearchCV(model, param_grid=params, scoring='neg_mean_squared_error', cv=5)
    grid_model.fit(X_features, y_target)
    rmse = np.sqrt(-1*grid_model.best_score_)
    print('{0} 5v 시 최적 평균 RMSE 값:{1}, 최적 alpha:{2}'.format(model.__class__.__name__, np.round(rmse, 4)

ridge_params = {'alpha':[0.05, 0.1, 1, 5, 8, 10, 12, 15, 20]}
lasso_params = {'alpha':[0.001, 0.005, 0.008, 0.05, 0.03, 0.1, 0.5, 1, 5, 10]}
print_best_params(ridge_reg, ridge_params)
print_best_params(lasso_reg, lasso_params)
```

Ridge 5v 시 최적 평균 RMSE 값:0.1418, 최적 alpha:{'alpha': 12}
Lasso 5v 시 최적 평균 RMSE 값:0.142, 최적 alpha:{'alpha': 0.001}

- train_test_split 말고 X_features와 y_target을 5개 교차 검증 fold 세트로 분할해 평균 RMSE 측정
 - cross_val_score() 이용
- 5개 fold 세트로 학습한 후 평가해도 여전히 라쏘가 릿지 모델보다 성능 낮음
- 릿지와 라쏘 모델에 대해 alpha 하이퍼 파라미터를 변화시키면서 최적 값 도출
- 모델별 최적화 하이퍼 파라미터 작업을 반복적으로 진행

- 별도의 함수를 생성
- `print_best_params(model, params)`
 - 모델과 하이퍼 파라미터 딕셔너리 객체를 받아 최적화 작업의 결과를 표시하는 함수
 - 이 함수 이용해 릿지 모델과 라쏘 모델의 최적화 alpha 값 추출
- 릿지 모델의 경우 alpha가 12에서 최적 평균 RMSE가 0.1418
- 라쏘 모델의 경우 alpha가 0.001에서 최적 평균 RMSE가 0.142
 - 알파값 최적화 이후 예측 성능 많이 좋아짐

```
lr_reg = LinearRegression()
lr_reg.fit(X_train, y_train)
ridge_reg = Ridge(alpha=12)
ridge_reg.fit(X_train, y_train)
lasso_reg = Lasso(alpha=0.001)
lasso_reg.fit(X_train, y_train)
```

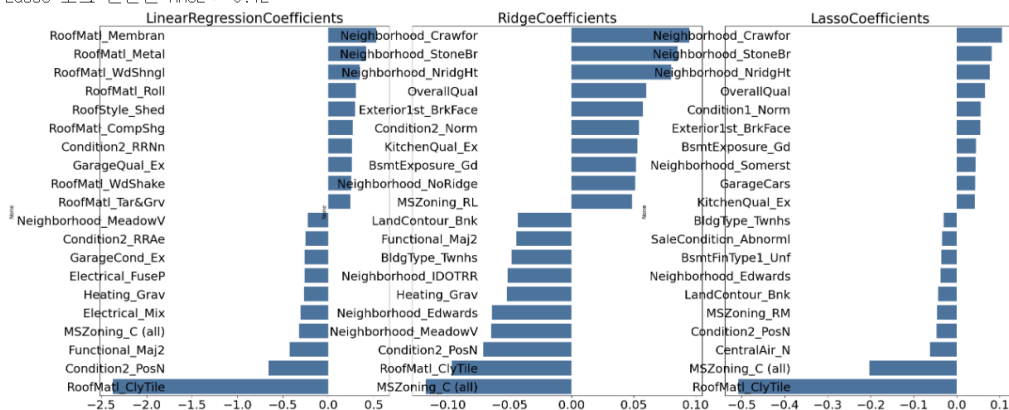
```
models = [lr_reg, ridge_reg, lasso_reg]
get_rmse(models)
```

```
models = [lr_reg, ridge_reg, lasso_reg]
visualize_coefficient(models)
```

LinearRegression 로그 변환된 RMSE : 0.132

Ridge 로그 변환된 RMSE : 0.124

Lasso 로그 변환된 RMSE : 0.12



- 선형 모델에 최적 alpha 값 설정한 뒤 `train_test_split()`으로 분할된 학습 데이터와 테스트 데이터 이용해 모델 학습/예측/평가
- alpha 값 최적화 후 테스트 데이터 세트의 예측 성능 더 좋아짐
- 모델별 회귀 계수도 많이 달라짐
- 기존과 달린 릿지와 라쏘 모델에서 비슷한 feature의 회귀 계수가 높음

- But, 라쏘 모델은 릿지에 비해 동일한 feature라도 회귀 계수의 값이 상당히 작음

```
from scipy.stats import skew

features_index = house_df.dtypes[house_df.dtypes != 'object'].index
skew_features = house_df[features_index].apply(lambda x: skew(x))
skew_features_top = skew_features[skew_features>1]

print(skew_features_top.sort_values(ascending=False))
```

```
MiscVal      24.451640
PoolArea     14.813135
LotArea      12.195142
3SsnPorch    10.293752
LowQualFinSF  9.002080
KitchenAbvGr  4.483784
BsmtFinSF2    4.250888
ScreenPorch   4.117977
BsmtHalfBath  4.099186
EnclosedPorch 3.086696
MasVnrArea    2.673661
LotFrontage   2.382499
OpenPorchSF   2.361912
BsmtFinSF1    1.683771
WoodDeckSF    1.539792
TotalBsmtSF   1.522688
MSSubClass    1.406210
1stFlrSF      1.375342
GrLivArea     1.365156
dtype: float64
```

- 사이파이 stats 모듈의 skew() 함수를 이용해 칼럼의 데이터 세트의 왜곡된 정도 추출 가능
- DataFrame에서 숫자형 feature의 왜곡 정도 확인
- 일반적으로 skew() 함수의 반환 값이 1 이상인 경우를 왜곡 정도가 높다고 판단
 - 상황에 따라 편차 존재
 - 여기서는 1 이상의 값을 반환하는 feature만 추출해 왜곡 정도를 완화하기 위해 로그 변환 적용

- 숫자형 feature 칼럼 index 객체를 추출해 구한 숫자형 칼럼 데이터 세트의 apply lambda 식 skew()를 호출해 숫자형 feature의 왜곡 정도를 구함
 - 주의점은 skew()를 적용하는 숫자형 feature에서 원-핫 인코딩된 카테고리 숫자형 feature는 제외해야 함
 - 카테고리 feature는 코드성 feature이므로 인코딩 시 왜곡될 가능성 높음
 - ⇒ skew() 함수 적용하는 DataFrame은 원-핫 인코딩 적용된 house_df_ohe가 아니라 적용 안된 house_df이어야 함

```
house_df[skew_features_top.index] = np.log1p(house_df[skew_features_top.index])
```

제한된 코드에 라이선스가 적용될 수 있습니다. | Yanghojun/yanghojun.github.io | evelyn82ny/Machine-learning | tjdgm1077/book_datasets

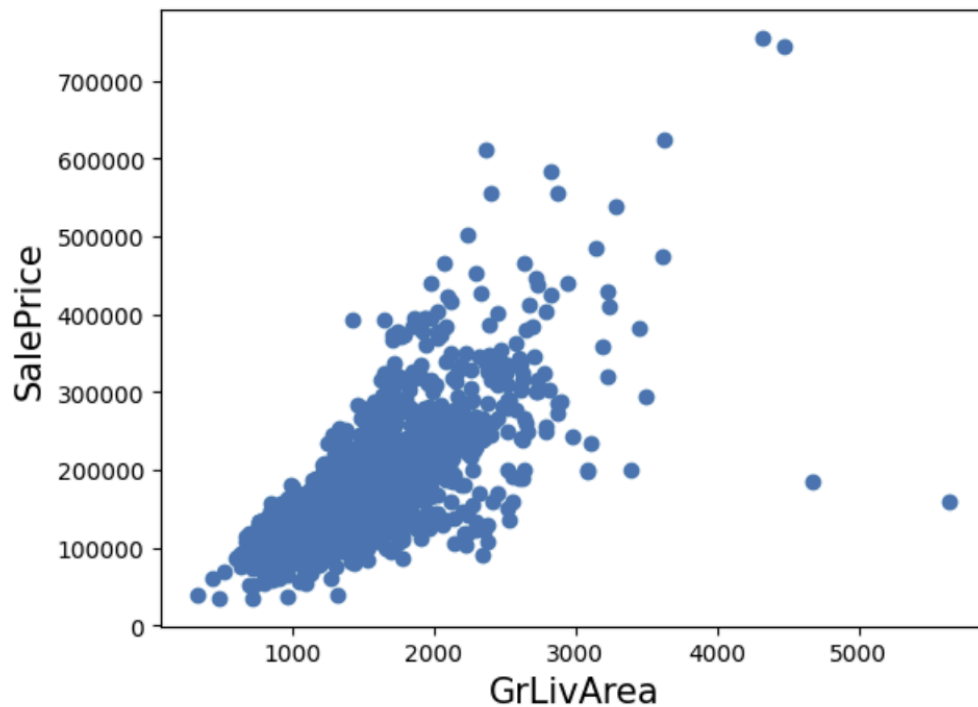
```
house_df_ohe = pd.get_dummies(house_df)
y_target = house_df_ohe['SalePrice']
X_features = house_df_ohe.drop('SalePrice', axis=1, inplace=False)
X_train, X_test, y_train, y_test = train_test_split(X_features, y_target, test_size=0.2, random_state=156)

ridge_params = {'alpha':[0.05, 0.1, 1, 5, 8, 10, 12, 15, 20]}
lasso_params = {'alpha':[0.001, 0.005, 0.008, 0.05, 0.03, 0.1, 0.5, 1, 5, 10]}
print_best_params(ridge_reg, ridge_params)
print_best_params(lasso_reg, lasso_params)
```

Ridge 5v 시 최적 평균 RMSE 값:0.1275, 최적 alpha:{'alpha': 10}
Lasso 5v 시 최적 평균 RMSE 값:0.1252, 최적 alpha:{'alpha': 0.001}

- 로그 변환 후 feature들의 왜곡 정도를 다시 확인해보면 여전히 높은 왜곡 정도 가진 feature 존재
 - 더 이상 로그 변환 하지 않더라도 개선하기 어렵기 때문에 그냥 유지
- house_df의 feature를 일부 로그 변환했으므로 다시 원-핫 인코딩 적용한 house_df_ohe 만듦
 - 이에 기반한 feature 데이터 세트와 target 데이터 세트, 학습/테스트 데이터 세트 모두 다시 만듦
- 여기에 앞에서 생성한 print_best_params() 함수 이용해 최적 alpha 값과 RMSE 출력
- 두 모델 모두 feature의 로그 변환 이전과 비교해 5 fold 교차 검증 평균 RMSE 값 향상
 - 릿지 경우 0.1418 → 0.1275
 - 라쏘 경우 0.142 → 0.1252

```
plt.scatter(x=house_df_org['GrLivArea'], y=house_df_org['SalePrice'])
plt.ylabel('SalePrice', fontsize=15)
plt.xlabel('GrLivArea', fontsize=15)
plt.show()
```



- 이상치 데이터 분석
- 회귀 계수 높은 feature = 예측에 많은 영향 미치는 중요 feature의 이상치 데이터 처리가 중요
- 세 개 모델 모두에서 가장 큰 회귀 계수 가지는 GrLivArea feature 데이터 분포
- GrLivArea와 SalePrice의 관계 시각화
 - 양의 상관도가 매우 높음
 - 오른쪽 아래 두 개의 데이터는 일반적인 관계에서 너무 어긋나 있음
 - 이상치로 간주하고 삭제

```
cond1 = house_df_ohe['GrLivArea']>np.log1p(4000)
cond2 = house_df_ohe['SalePrice']<np.log1p(500000)
outlier_index = house_df_ohe[cond1 & cond2].index

print('이상치 레코드 index:', outlier_index.values)
print('이상치 삭제 전 house_df_ohe shape:', house_df_ohe.shape)

house_df_ohe.drop(outlier_index, axis=0, inplace=True)
print('이상치 삭제 후 house_df_ohe shape:', house_df_ohe.shape)
```

이상치 레코드 index: [523 1298]
 이상치 삭제 전 house_df_ohe shape: (1460, 270)
 이상치 삭제 후 house_df_ohe shape: (1458, 270)

```
제안된 코드에 라이선스가 적용될 수 있습니다. | Yanghojun/yanghojun.github.io | evelyn82ny/Machine-learning
y_target = house_df_ohe['SalePrice']
X_features = house_df_ohe.drop('SalePrice', axis=1, inplace=False)
X_train, X_test, y_train, y_test = train_test_split(X_features, y_target, test_size=0.2, random_state=156)

ridge_params = {'alpha':[0.05, 0.1, 1, 5, 8, 10, 12, 15, 20]}
lasso_params = {'alpha':[0.001, 0.005, 0.008, 0.05, 0.03, 0.1, 0.5, 1, 5, 10]}

print_best_params(ridge_reg, ridge_params)
print_best_params(lasso_reg, lasso_params)
```

Ridge 5v 시 최적 평균 RMSE 값:0.1125, 최적 alpha:{'alpha': 8}
 Lasso 5v 시 최적 평균 RMSE 값:0.1122, 최적 alpha:{'alpha': 0.001}

- 데이터 변환 모두 완료된 house_df_ohe에서 대상 데이터 필터링
- GrLivArea와 SalePrice 모두 로그 변환됐으므로 반영한 조건 생성
 - 불린 인덱싱으로 대상 찾을
 - 찾은 데이터의 DataFrame 인덱스와 drop() 이용하여 해당 데이터 삭제
- 데이터 1460→1458개로 줄어듦
- 업데이트된 house_df_ohe를 기반으로 feature 데이터 세트와 타깃 데이터 세트 다시 생성, print_best_params() 함수 이용
- 단 두 개의 이상치 데이터만 제거했는데 예측 수치 매우 크게 향상됨
- 릿지 경우
 - 최적 alpha 값 12 → 8
 - 평균 RMSE 0.1275 → 0.1125
- 라쏘 경우
 - 평균 RMSE 0.1252 → 0.1122
- 웬만큼 하이퍼 파라미터 튜닝해도 어려운 수치 개선
- → GrLivArea 속성이 회귀 모델에서 차지하는 영향도가 큼

제한된 코드에 라이선스가 적용될 수 있습니다.

```
from xgboost import XGBRegressor

xgb_params = {'n_estimators': [1000]}
xgb_reg = XGBRegressor(n_estimators=1000, learning_rate=0.05, colsample_bytree=0.5, subsample=0.8)
print_best_params(xgb_reg, xgb_params)
```

XGBRegressor 5v 시 최적 평균 RMSE 값: 0.1206, 최적 alpha: {'n_estimators': 1000}

```
from lightgbm import LGBMRegressor

lgbm_params = {'n_estimators': [1000]}
lgbm_reg = LGBMRegressor(n_estimators=1000, learning_rate=0.05, num_leaves=4, subsample=0.6, colsample_bytree=0.5)
print_best_params(lgbm_reg, lgbm_params)
```

- 회귀 트리 이용해 회귀 모델 생성
- XGBoost 회귀 트리 적용 시 5fold 세트 평균 RMSE가 약 0.1178
- LightGBM 적용 시 5 fold 세트 평균 RMSE가 약 0.1163

```
def get_rmse_pred(preds):
    for key in preds.keys():
        pred_value = preds[key]
        mse = mean_squared_error(y_test, pred_value)
        rmse = np.sqrt(mse)
        print('{0} 모델의 RMSE: {1}'.format(key, rmse))
```

```
ridge_reg = Ridge(alpha=8)
ridge_reg.fit(X_train, y_train)
lasso_reg = Lasso(alpha=0.001)
lasso_reg.fit(X_train, y_train)

ridge_pred = ridge_reg.predict(X_test)
lasso_pred = lasso_reg.predict(X_test)

pred = 0.4 * ridge_pred + 0.6 * lasso_pred
preds = {'최종혼합': pred, 'Ridge': ridge_pred, 'Lasso': lasso_pred}
get_rmse_pred(preds)
```

최종혼합 모델의 RMSE: 0.10006075517615193
Ridge 모델의 RMSE: 0.10340697165289348
Lasso 모델의 RMSE: 0.10024171179335342

```
xgb_reg = XGBRegressor(n_estimators=1000, learning_rate=0.05, colsample_bytree=0.5, subsample=0.8)
lgbm_reg = LGBMRegressor(n_estimators=1000, learning_rate=0.05, num_leaves=4, subsamples=0.6, colsample_bytree=0.5)

xgb_reg.fit(X_train, y_train)
lgbm_reg.fit(X_train, y_train)
xgb_pred = xgb_reg.predict(X_test)
lgbm_pred = lgbm_reg.predict(X_test)

pred = 0.5 * xgb_pred + 0.5 * lgbm_pred
preds = {'최종혼합': pred, 'XGB': xgb_pred, 'LGBM': lgbm_pred}
get_rmse_pred(preds)
```

- 개별 회귀 모델의 예측 결과값을 혼합해 최종 회귀 값을 예측

- 앞서 구한 릿지, 라쏘 모델을 혼합
- 최종 혼합 모델, 개별 모델의 RMSE 값 출력하는 get_rmse_pred() 함수 생성
- 각 모델의 예측값을 계산한 뒤 개별 모델과 최종 혼합 모델의 RMSE 구함
- 최종 혼합 모델의 RMSE가 개별 모델보다 성능 약간 개선됨
- 릿지 모델 예측값에 0.4, 라쏘 모델 예측값에 0.6 곱한 뒤 더함
- XGBoost와 LightGBM 혼합한 결과
 - 개별 모델의 RMSE보다 조금 향상됨

```
from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_error

def get_stacking_base_datasets(model, X_train_n, y_train_n, X_test_n, n_folds):
    kf = KFold(n_splits=n_folds, shuffle=False)
    train_fold_pred = np.zeros((X_train_n.shape[0], 1))
    test_pred = np.zeros((X_test_n.shape[0], n_folds))
    print(model.__class__.__name__, 'model 시작')

    for folder_counter, (train_index, valid_index) in enumerate(kf.split(X_train_n)):
        print('### 폴드 세트: ', folder_counter, '시작')
        X_tr = X_train_n[train_index]
        y_tr = y_train_n[train_index]
        X_te = X_train_n[valid_index]

        model.fit(X_tr, y_tr)

        train_fold_pred[valid_index, :] = model.predict(X_te).reshape(-1, 1)
        test_pred[:, folder_counter] = model.predict(X_test_n)

    test_pred_mean = np.mean(test_pred, axis=1).reshape(-1, 1)

    return train_fold_pred, test_pred_mean
```

```
y_train_n = y_train.values

ridge_train, ridge_test = get_stacking_base_datasets(ridge_reg, X_train.values, y_train_n, X_test.values, 5)
lasso_train, lasso_test = get_stacking_base_datasets(lasso_reg, X_train.values, y_train_n, X_test.values, 5)
xgb_train, xgb_test = get_stacking_base_datasets(xgb_reg, X_train.values, y_train_n, X_test.values, 5)
lgbm_train, lgbm_test = get_stacking_base_datasets(lgbm_reg, X_train.values, y_train_n, X_test.values, 5)
```

- 스택킹 모델은 두 종류의 모델 필요
 - 개별적인 기반 모델
 - 이 개별 기반 모델의 예측 데이터를 학습 데이터로 만들어서 학습하는 최종 메타 모델
- 핵심은 여러 개별 모델의 예측 데이터를 각각 스택킹 형태로 결합

- 최종 메타 모델의 학습용 feature 데이터 세트와 테스트용 feature 데이터 세트 만드는 것
- `get_stacking_base_datasets()`는 인자로 개별 기반 모델, 원래 사용되는 학습 데이터와 테스트용 feature 데이터를 입력받음
- 함수 내 개별 모델이 K-Fold 세트로 설정된 fold 세트 내부에서 원본 학습 데이터를 다시 추출해 학습과 예측 수행한 뒤 결과 저장
- 저장된 예측 데이터는 추후 메타 모델의 학습 feature 데이터 세트로 이용됨
- 함수 내에서 fold 세트 내부 학습 데이터로 학습된 개별 모델이 인자로 입력된 원본 테스트 데이터를 예측한 뒤, 예측 결과를 평균하여 테스트 데이터로 생성
- `get_stacking_base_datasets()`를 모델별로 적용
 - 메타 모델이 사용할 학습 feature 데이터 세트와 테스트 feature 데이터 세트를 추출
 - 릿지, 라쏘, XGBoost, LightGBM 에 적용

```
Stack_final_X_train = np.concatenate((ridge_train, lasso_train, xgb_train, lgbm_train), axis=1)
Stack_final_X_test = np.concatenate((ridge_test, lasso_test, xgb_test, lgbm_test), axis=1)

meta_model_lasso = Lasso(alpha=0.0005)
meta_model_lasso.fit(Stack_final_X_train, y_train)
final = meta_model_lasso.predict(Stack_final_X_test)
mse = mean_squared_error(y_test, final)
rmse = np.sqrt(mse)

print('스태킹 회귀 모델의 최종 RMSE 값은:', rmse)
```

스태킹 회귀 모델의 최종 RMSE 값은: 0.09704503149148055

- 각 개별 모델이 반환하는 학습용 feature 데이터와 테스트용 feature 데이터 세트를 결합해 최종 메타 모델에 적용
- 최종 스태킹 회귀 모델 적용 결과, 테스트 데이터 세트에서 RMSE가 현재까지 가장 좋은 성능 평가 보여줌

